SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
PRACTICAL ASSIGNMENT- 4

1. Student Details

Student Name	Homesh Sawarkar
PRN	202501110069
Division	A
Batch	A-2
Roll No.	202501110069
Date of Submission	11-10-2026

2. Practical Details

Practical No.	4
Practical Title	References and Pointers in C++; new and delete Operators for Dynamic MemoryAllocation
Date Performed	09-09-2026
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement
(a) Write a C++ program that demonstrates the use of pointers and references: declaring them, accessing and modifying a variable through them, and passing arguments to functions by value, by pointer and by reference.
(b) Write a C++ program that uses the new and delete operators to allocate and release memory dynamically for a single variable and for an array whose size is decided at run time.

4. Objectives

1. To understand the concept of pointers and references and the difference between them.
2. To understand call by value, call by pointer and call by reference.
3. To use the new and delete (delete[]) operators for dynamic memory allocation and deallocation.
4. To understand how to avoid memory leaks and dangling pointers.

5. Expected Learning Outcomes

- Declare and use pointers and references in C++.
- Pass arguments to functions in three different ways and observe the effect on the original data.
- Allocate and free memory at run time using new, new[], delete and delete[].
- Write safer programs by setting freed pointers to nullptr.

6. Theory / Concept
Pointer:A pointer isa variable that stores the memory address of another variable. It is declared with * (for example, int ptr = &x;). The address-of operator (&) gives the address of a variable and the dereference operator () accesses the value stored at that address. A pointer can be reassigned, can be nullptr, and supports pointer arithmetic.
Reference: A reference is an alias (another name) for an existing variable. It is declared with & (for example, int &ref = x;). A reference must be initialized when it is declared, cannot be reseated to refer to another variable, and cannot be null. Any change made through the reference changes the original variable.

Call by value, pointer and reference: In call by value a copy of the argument is passed, so the original is not changed. In call by pointer the address is passed and the function changes the original through the dereference operator. In call by reference the function receives an alias of the argument and changes the original directly with simpler syntax.

Dynamic memory allocation: Memory for local variables is allocated on the stack automatically. Memory that must be created at run time, or must live beyond a function's scope, is allocated on the heap (free store) using the new operator. new returns the address of the allocated memory. For a single variable the syntax is int *p = new int(25); and for an array int *arr = new int[n];.

delete and delete[]: Memory obtained with new must be released with delete, and memory obtained with new[] must be released with delete[]. If memory is not released, a memory leak occurs. After deletion the pointer becomes a dangling pointer, so it is good practice to set it to nullptr.

7. Algorithm / Pseudocode

1. START
2. Declare an integer x and initialize it with 10.
3. Create a pointer ptr that stores the address of x and a reference ref that is an alias of x.
4. Display x, *ptr and ref, and check that ptr == &x and &ref == &x.
5. Change x through the pointer (*ptr = 30) and through the reference (ref = 40), and display the values.
6. Declare p = 5 and q = 9.
7. Call swapByValue(p, q) and display p and q (unchanged).
8. Call swapByPointer(&p, &q) and display p and q (swapped).
9. Call swapByReference(p, q) and display p and q (swapped back).
10. STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
ProgrammingLanguage	C++
IDE / Platform	Visual Studio Code (VS Code)
Libraries / Packages	iostream, string

09. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	
Repository Name	**Assigenment4**
Branch Used	**main**
Commit ID / Latest Commit	**Add file via upload**

Particular	Details
Date of Last Commit	11-10-2026

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
1	n = 3; numbers 10 20 30	x changes to 20 then 30; swaps give a = 9, b = 5 then a = 5, b = 9; dynamic number = 25; Sum = 60; memory freed	Same as expected	Pass
2	n = 5; numbers 5 10 15 20 25	x changes to 20 then 30; swaps give a = 9, b = 5 then a = 5, b = 9; dynamic number = 25; Sum = 75; memory freed	Same as expected	Pass

```
PS C:\Users\Homesh\Desktop\20240110075_Project files> cd "c:\Users\Homesh" ; if ($?) { g++ assi4.cpp -o assi4 } ; if ($?) {
.\assi4 } 0 };
--- Part A: Pointer and Reference ---
x = 10
Value using pointer = 10
Value using reference = 10
x after changing with pointer = 20
x after changing with reference = 30

Before swap: a = 5, b = 9
After swapByPointer: a = 9, b = 5
After swapByReference: a = 5, b = 9

--- Part B: new and delete ---
Dynamic number = 25
```

11. Results / Observations

The C++ program was implemented successfully in Visual Studio Code. In Part A, the pointer ptr and the reference ref both gave access to the variable x, and changing the value through either of them changed x itself. The swapByPointer() and swapByReference() functions swapped the values of a and b in the original variables. In Part B, memory was allocated at run time using new for a single variable and for an array whose size was entered by the user. The memory was released using delete and delete[], and the pointers were set to nullptr to avoid dangling pointers. Both test cases gave the expected output.

12. Conclusion

The practical successfully demonstrated pointers, references and the three ways of passing arguments to functions in C++. It also showed how the new and delete operators allocate and free memory dynamically, and why every allocation must be matched with a deallocation (and the pointer set to nullptr) to prevent memory leaks and dangling pointers.

13. Assignment Questions

Q1. What is the difference between a pointer and a reference in C++?

Answer : A pointerstores theaddress of a variable, can be reassigned to point to another variable, can be nullptr, and is accessed using the dereference operator (*). A reference is an alias for an existing variable, must be

initialized at the time of declaration, cannot be reassigned to refer to another variable, cannot be null, and is used like a normal variable. References give a simpler syntax, while pointers give more flexibility (pointer arithmetic, dynamic memory).

Q2. Explain the new and delete operators. Why is delete[] used?

Answer: The new operator allocates memory from the heap at run time and returns its address, for example int *p = new int; or int *arr = new int[n];. The delete operator releases memory allocated by new so that it can be reused. When an array is allocated with new[], it must be released with delete[] so that the whole array is freed properly (and destructors are called for every element in case of objects). Using delete instead of delete[] on an array causes undefined behaviour, and forgetting to delete causes a memory leak. Unlike malloc() and free(), new and delete are operators, do not need type casting, and call constructors and destructors.

14. Student Declaration

Ihereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work

Student Name	PRN	Signature
Homesh Sawarkar	202501110069	Homesh A.S